

Itinerary Audio Bot : Antigravity Guide



Ready to build with **Google Antigravity**? This walkthrough will help you get started step by step, installing Antigravity, opening your workspace, and watching an AI agent set up and run a complete project for you. The screenshots and visuals included come directly from the Antigravity environment, so you can follow along confidently without second-guessing any step.

What is Antigravity?

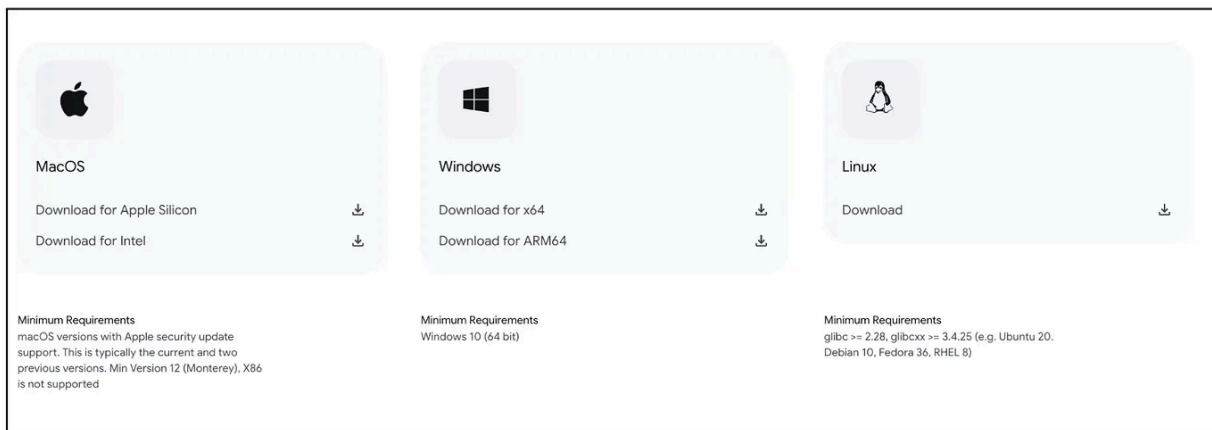
If you've used traditional IDEs like VS Code, you know they give you autocomplete suggestions but expect you to type each command yourself. Antigravity takes a different approach. Released in November 2025, it's a free, AI-powered development environment built around autonomous agents. You don't have to remember every command; instead, you describe the goal—"Install dependencies and run the app"—and an agent plans and executes the steps for you. Multiple agents can collaborate, and you can monitor them through the Agent Manager panel. Think of it as having a helpful assistant who reads documentation, runs commands and reports back, letting you focus on creativity.

Antigravity also includes a browser for live app previews and a terminal panel, so you can watch your bot come to life. It's perfect for demonstrations because learners see the environment being set up in real time while staying in control through review prompts.

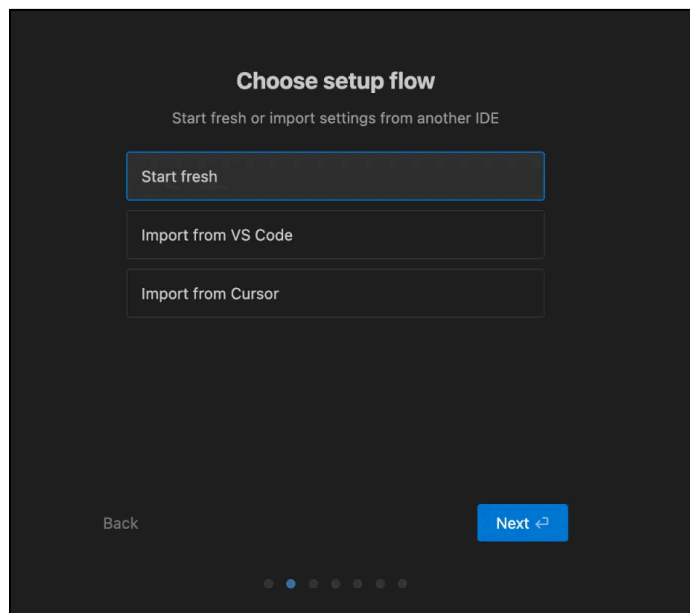
Installing Antigravity

Installing Antigravity is straightforward, and you can let the agent handle most of the heavy lifting. Follow these steps:

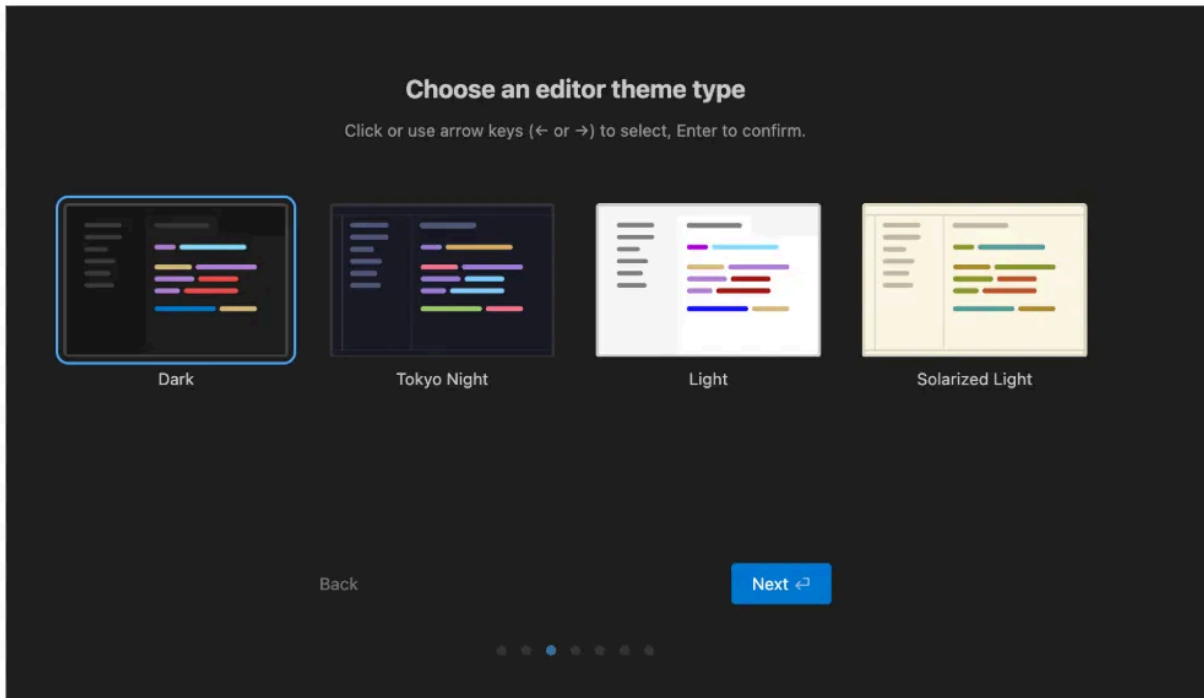
1. **Download the installer** – Visit the official downloads page at <https://antigravity.google> and pick the installer for your operating system. Run the installer.



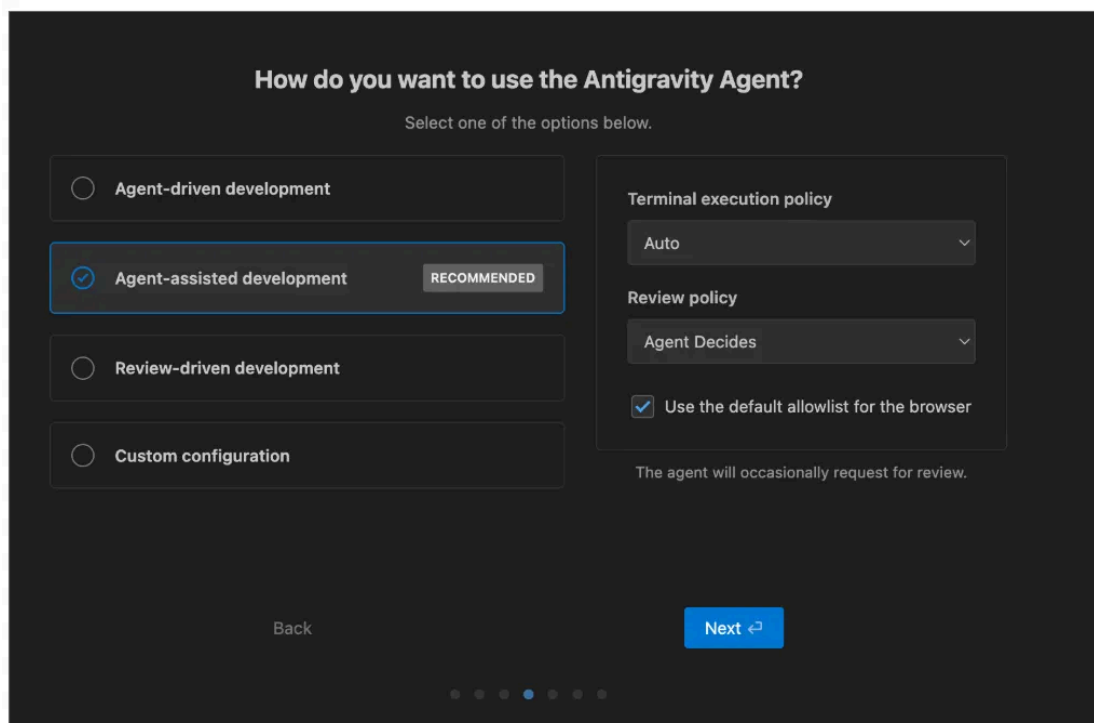
2. **Initial setup** – When the setup wizard appears: Choose “Start fresh” instead of importing VS Code settings.



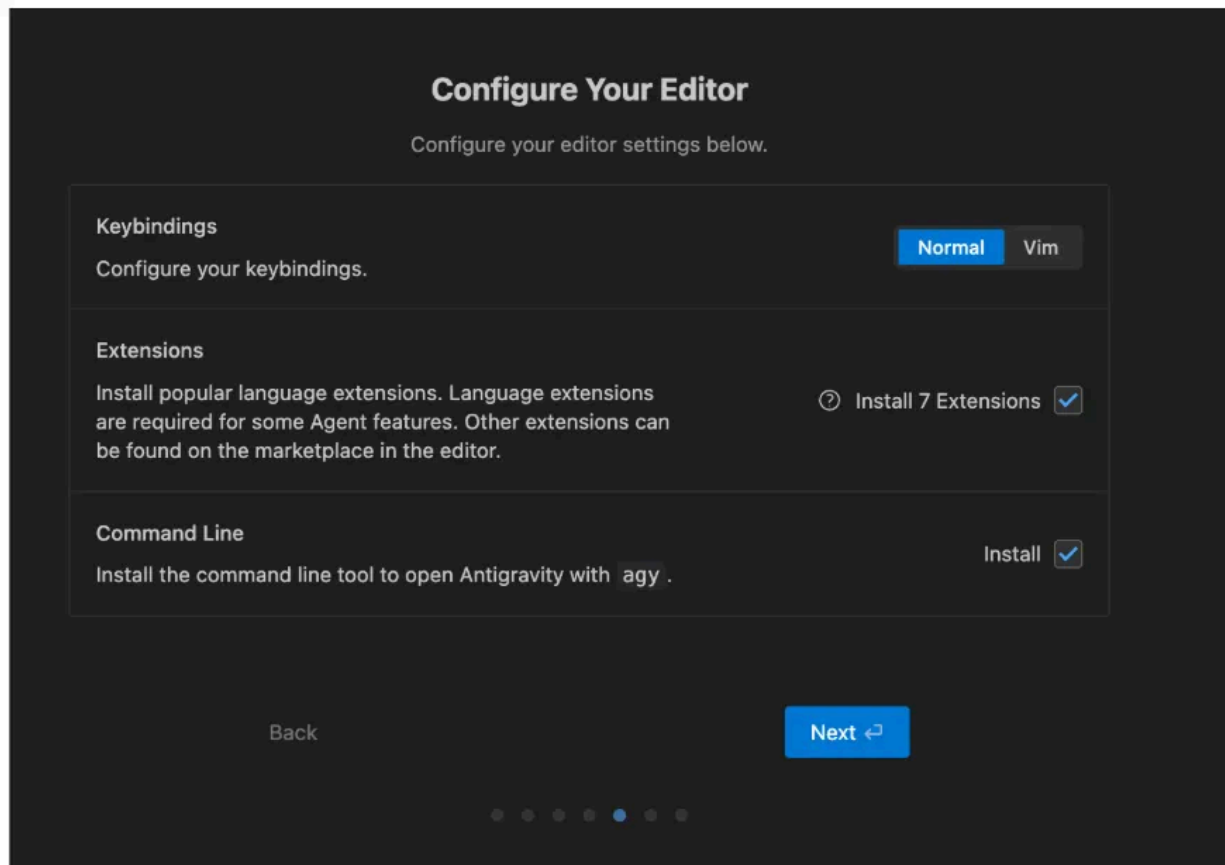
3. **Select a theme (dark or light)** – this just changes how the interface looks.



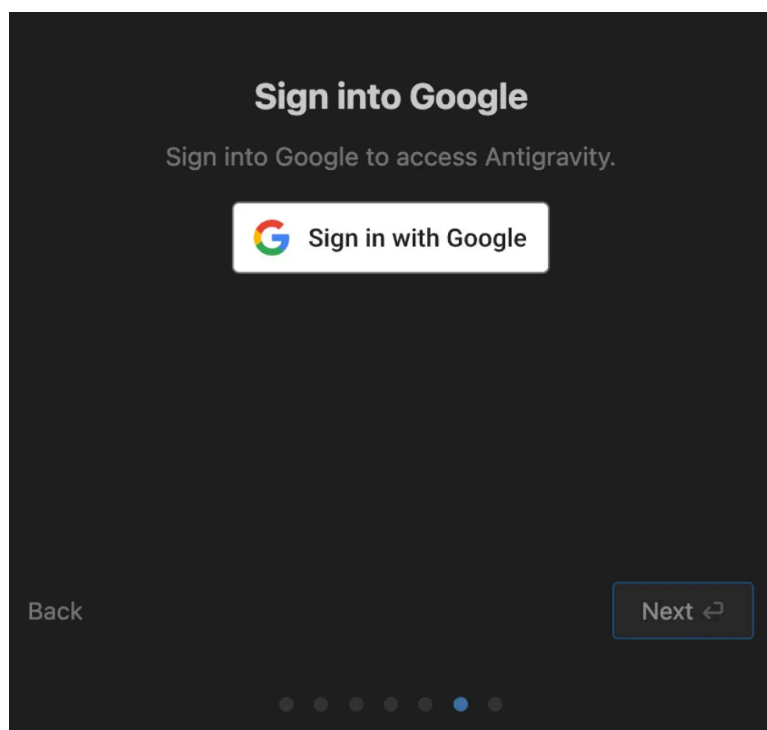
4. **Pick an agent policy.** The **review-driven** mode asks for confirmation before critical actions and is perfect for beginners.



5. **Configure your editor** – Antigravity offers optional extensions, keybindings and a command-line tool called `agy` . You can accept the defaults or customise them later.



6. **Sign in with your personal Google account**



7. The last step, as is typical, is the terms of use. You can make a decision if you'd like to opt-in or not and then click on Next.

Antigravity - Terms of Use

Antigravity is known to have certain security limitations. Users should be aware of potential risks, including data exfiltration and possible code execution. Avoid processing highly sensitive data and verify all the actions taken by the agent.

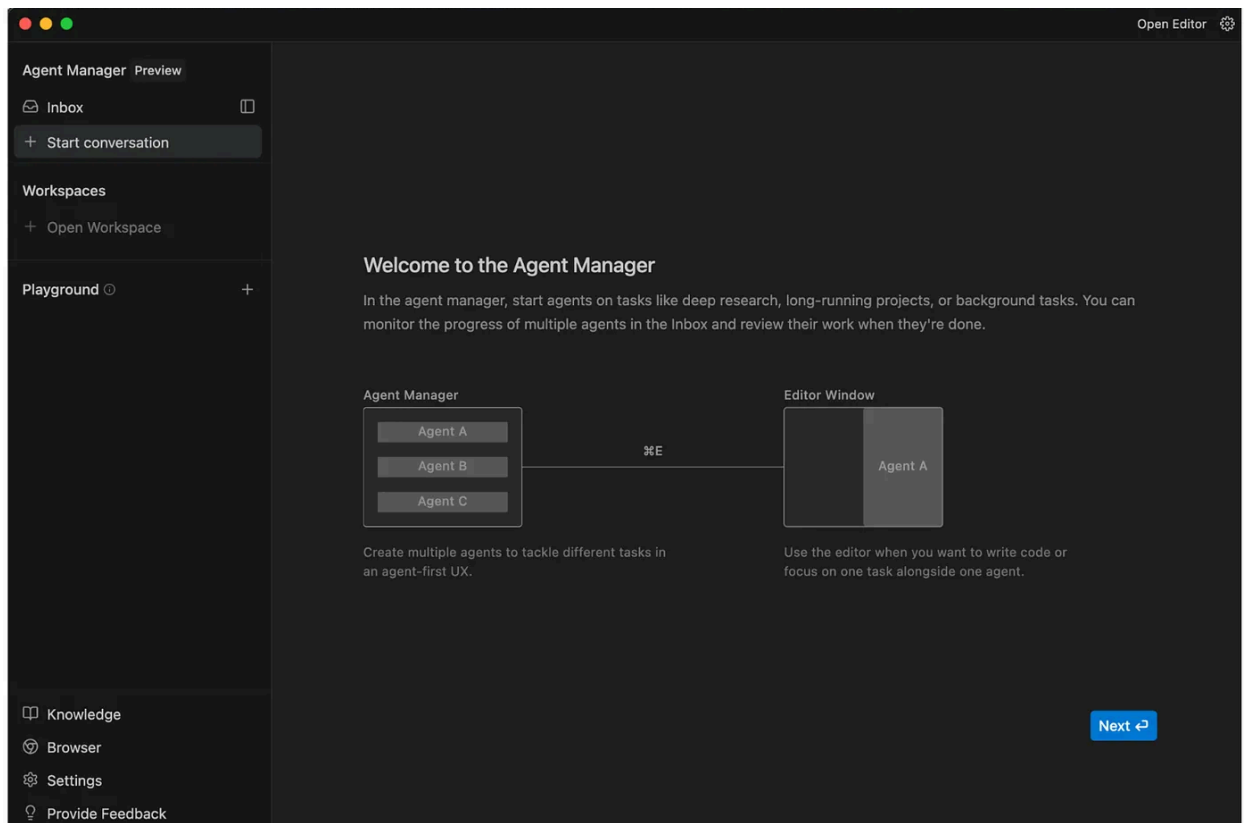
-
- ☒ Yes, I agree to allow Google to collect and use my Interactions data as defined in the [Google Antigravity Terms of Service](#) and consistent with the [Google Privacy Policy](#), for the purpose of evaluating, developing, and improving Google and Alphabet research, products, services, and machine learning technologies. I understand I can withdraw my consent at any time by visiting my account settings or e-mailing Antigravity Support.

Back

Next ↩



When installation is complete, Antigravity opens the Agent Manager window where you'll create tasks and monitor agents.



This interface acts as a **Mission Control dashboard**. It is designed for high-level orchestration, allowing developers to spawn, monitor, and interact with multiple agents operating asynchronously across different workspaces or tasks.

This architecture addresses a **key limitation** of previous **IDEs** that had more of a chatbot experience, which were linear and synchronous. In a traditional chat interface, the developer must wait for the AI to finish generating code before asking the next question. In Antigravity's Manager View, a developer can dispatch five different agents to work on five different bugs simultaneously, effectively multiplying their throughput.

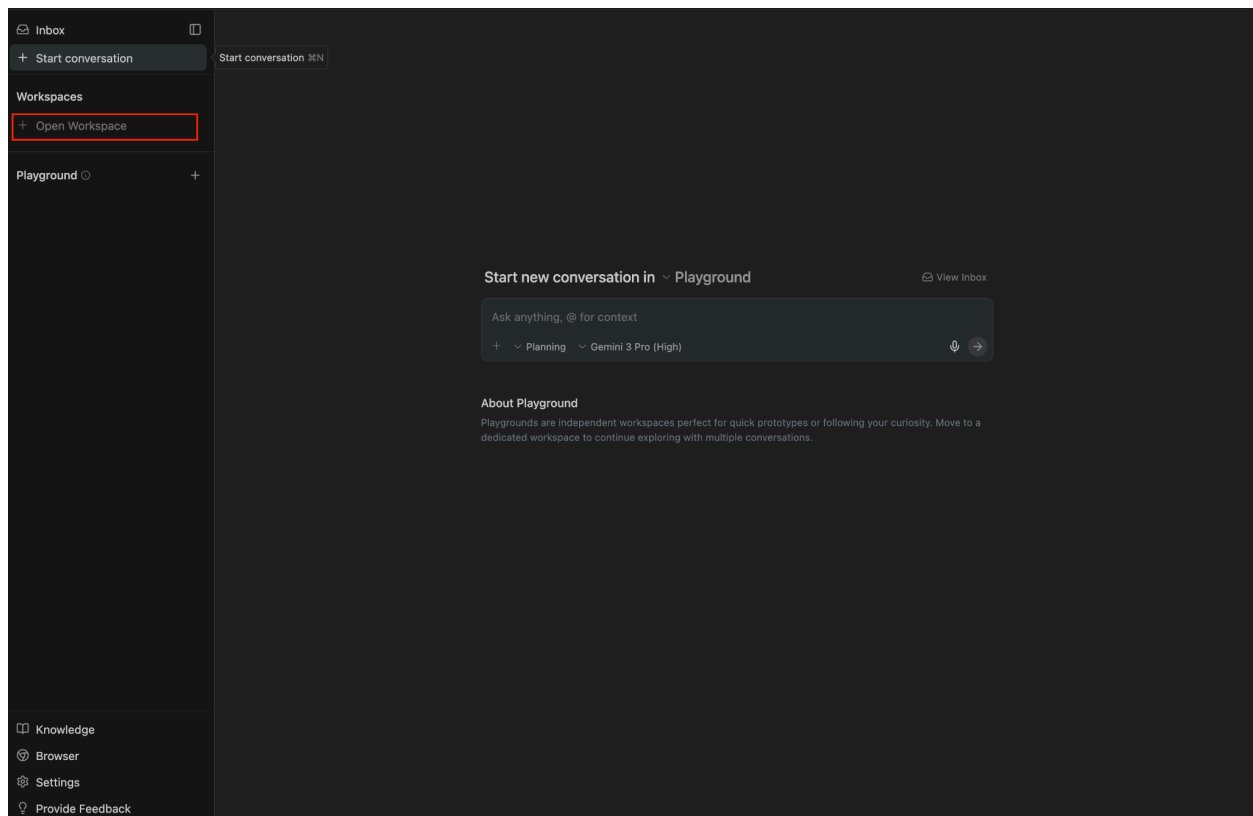
If you click on **Next** above, you have the option to open a Workspace.

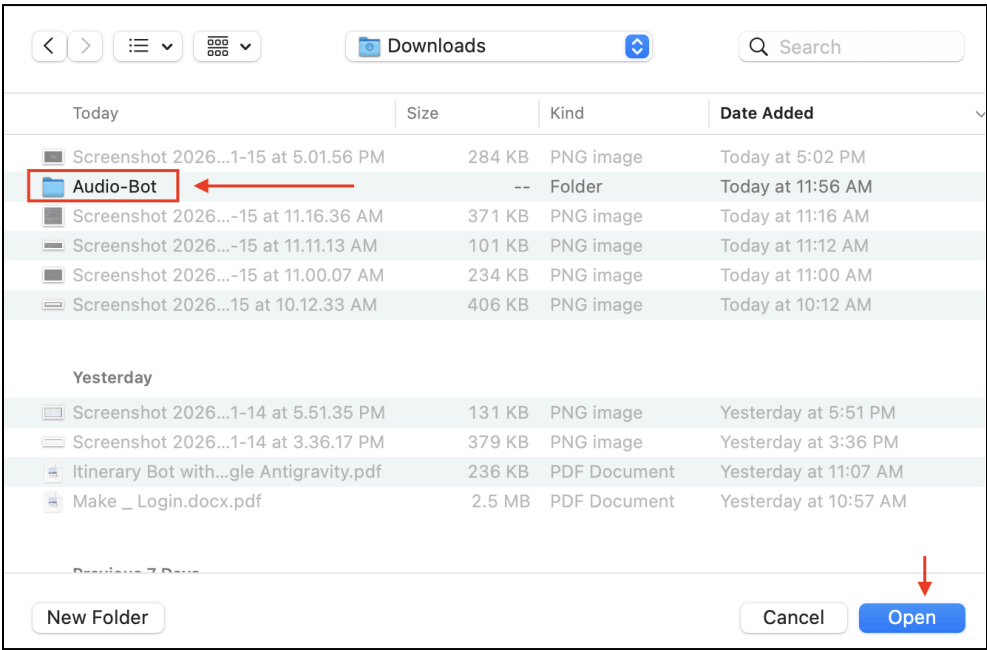
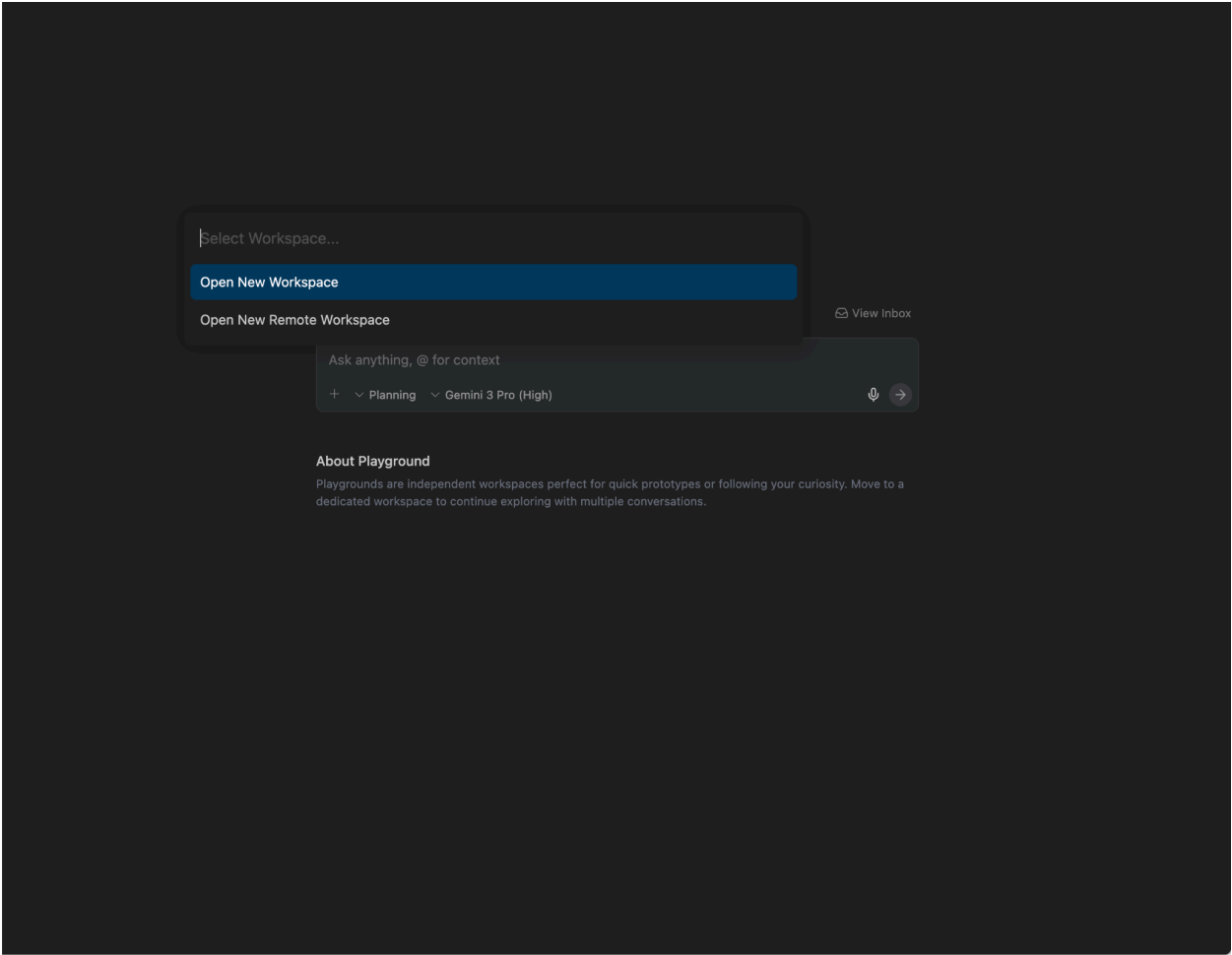
Project Walkthrough

Opening a workspace

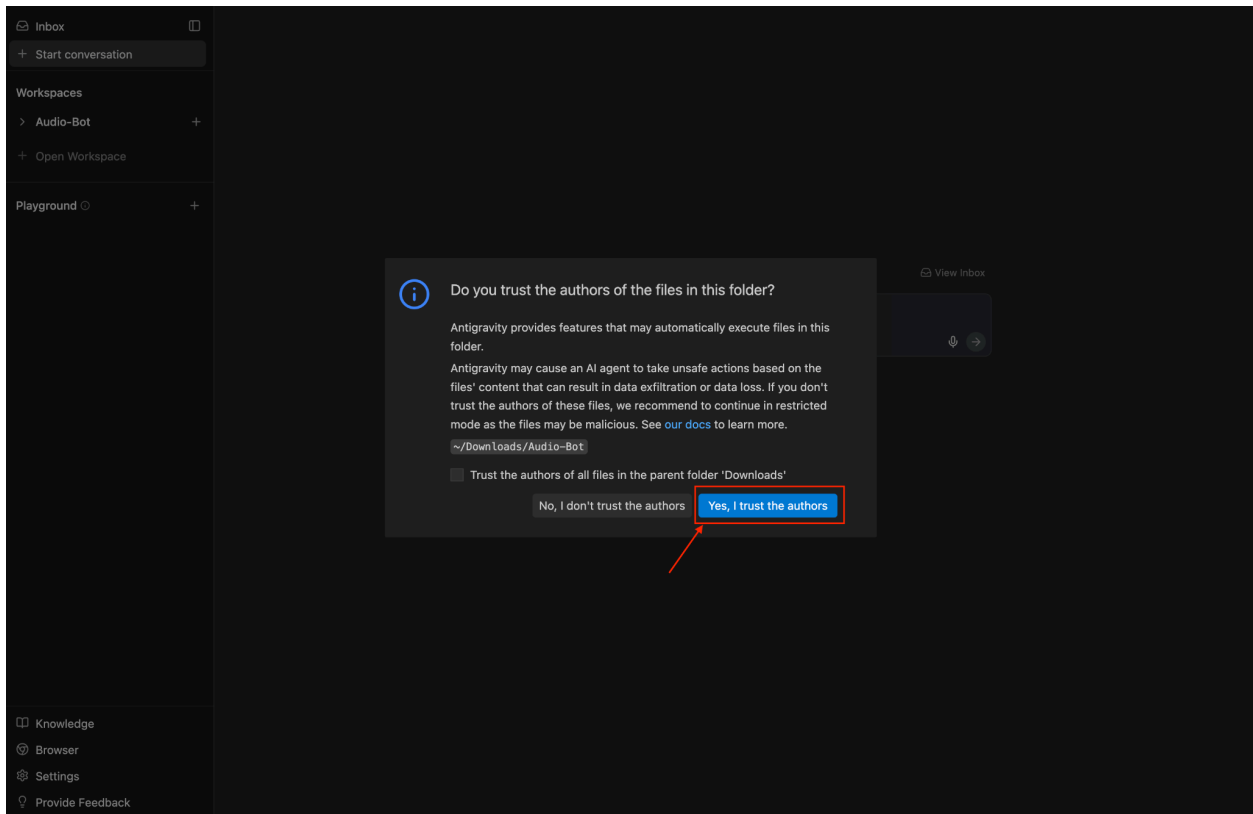
Antigravity organises your work in workspaces, similar to folders. To set up the Itinerary Audio Bot:

1. Download the project (.zip) from here.
Save it somewhere easy to find (for example: **Downloads**)
2. **Unzip the file** and place the unzipped folder in a **stable location**.
3. **Open Antigravity**, then click **“Open Workspace”** , Browse to the location where you placed the folder and open it. Antigravity will treat this folder as your **working directory**.

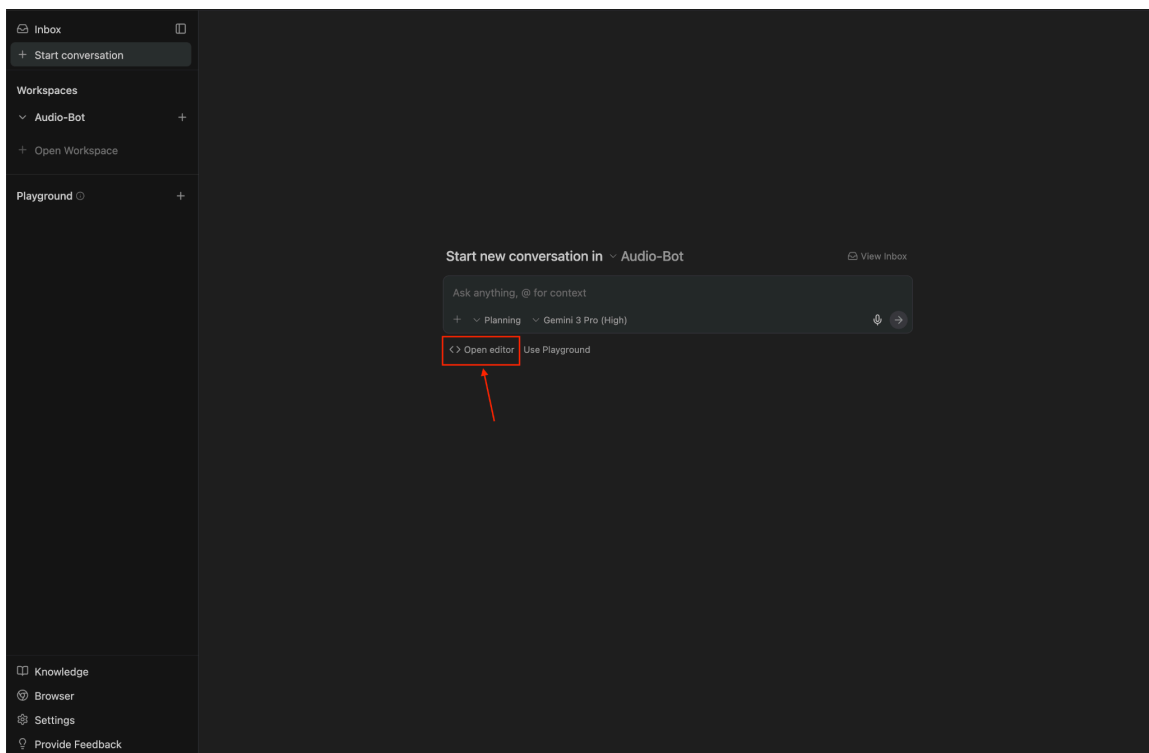




4. Antigravity will ask if you trust the authors of the files in this folder. Click **Yes, I trust the authors**. This is required so Antigravity can run code and let agents operate safely.

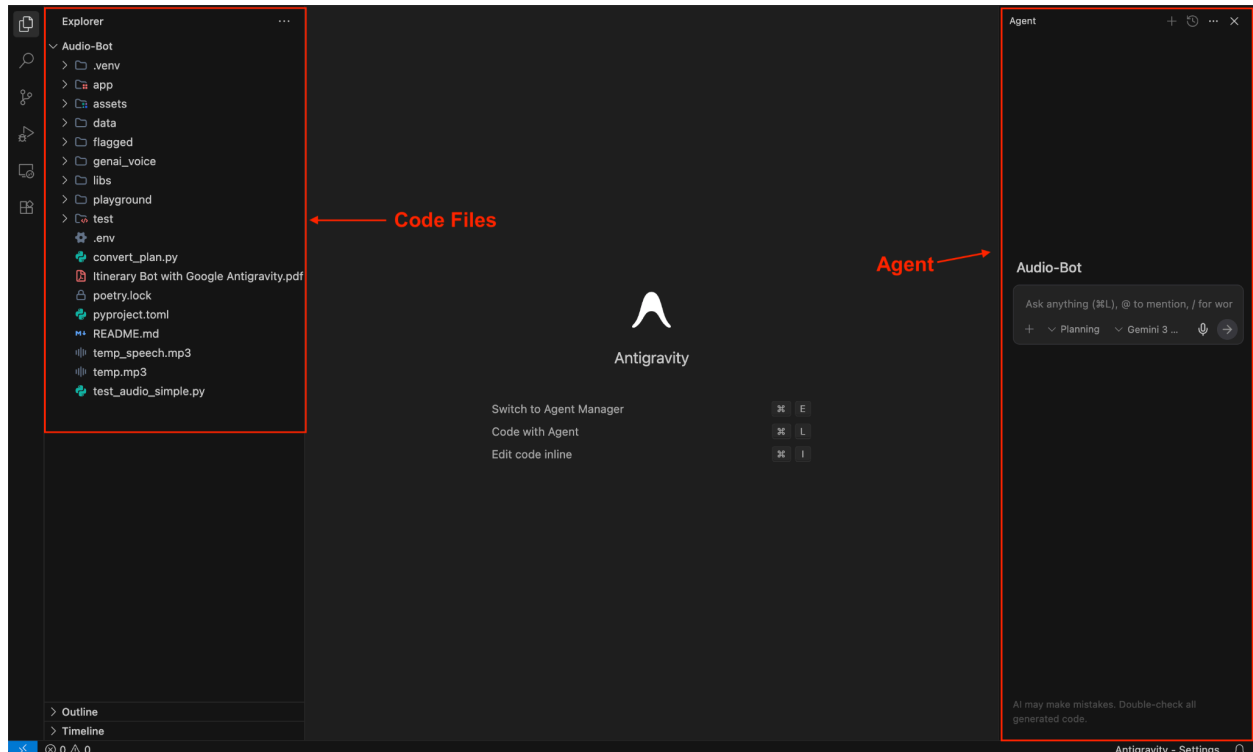


5. Once the workspace loads, click **Open editor** on the main screen. This opens the full coding interface (similar to VS Code).



6. Understanding the workspace layout :

- **Left panel (Code Files):** This shows all your project files and folders (code, configs, assets, README, etc.).
- **Right panel (Agent Area):** This is where you talk to Antigravity's AI agent and give it instructions.
- **Center area:** This is where files open when you view or edit code.

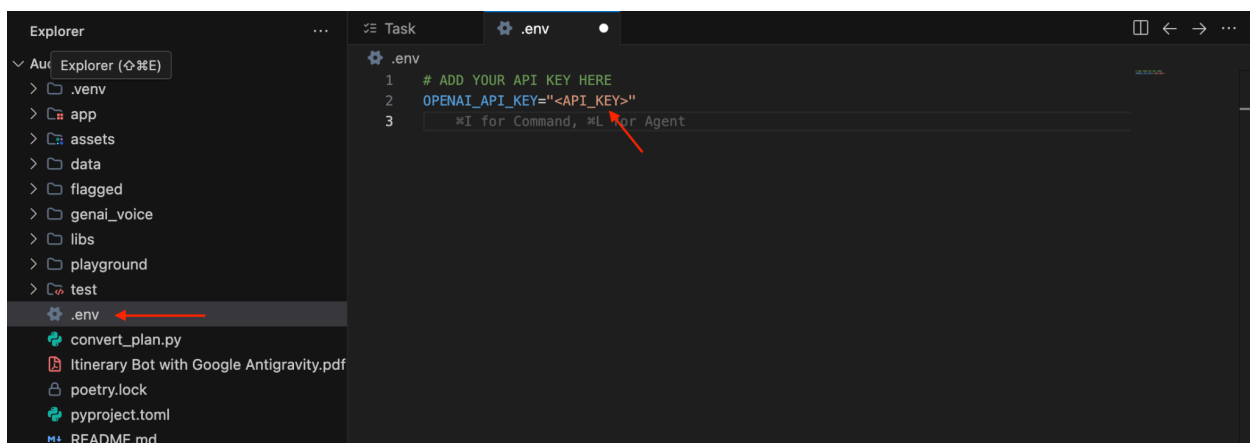


Add your OpenAI API key

In the **left-side file explorer**, click on the **.env** file. You'll see a line that looks like this:

```
OPENAI_API_KEY="<API_KEY>"
```

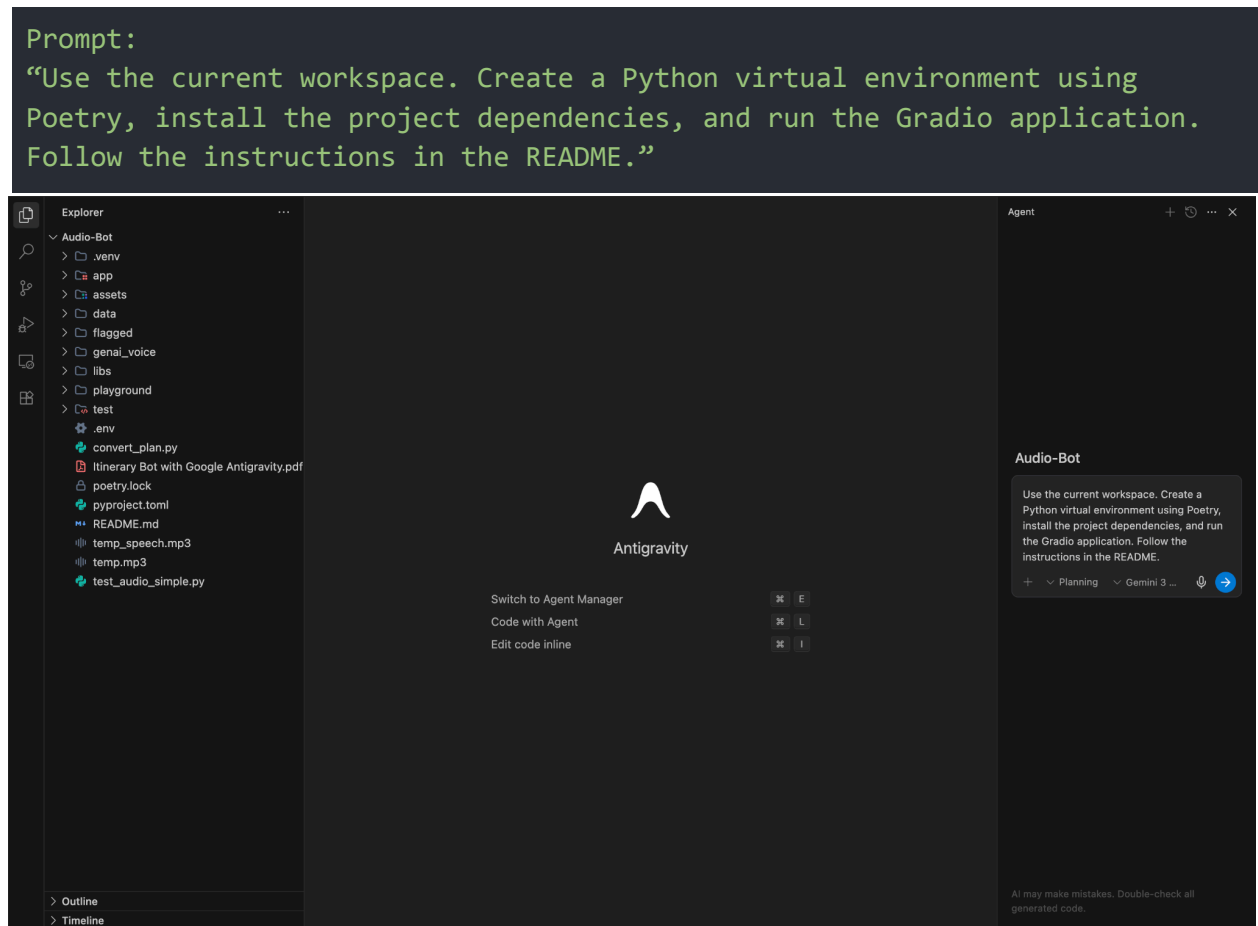
Replace **<API_KEY>** with **your own OpenAI API key & save (ctrl + s)**.



Let's set up the project with Antigravity Agent

Here's where Antigravity shines. Instead of manually setting up environment and running it, you'll delegate tasks to an agent:

1. Now, in the Agent area (right side), we'll write a single prompt that tells Antigravity to set up the entire project and run it for us.

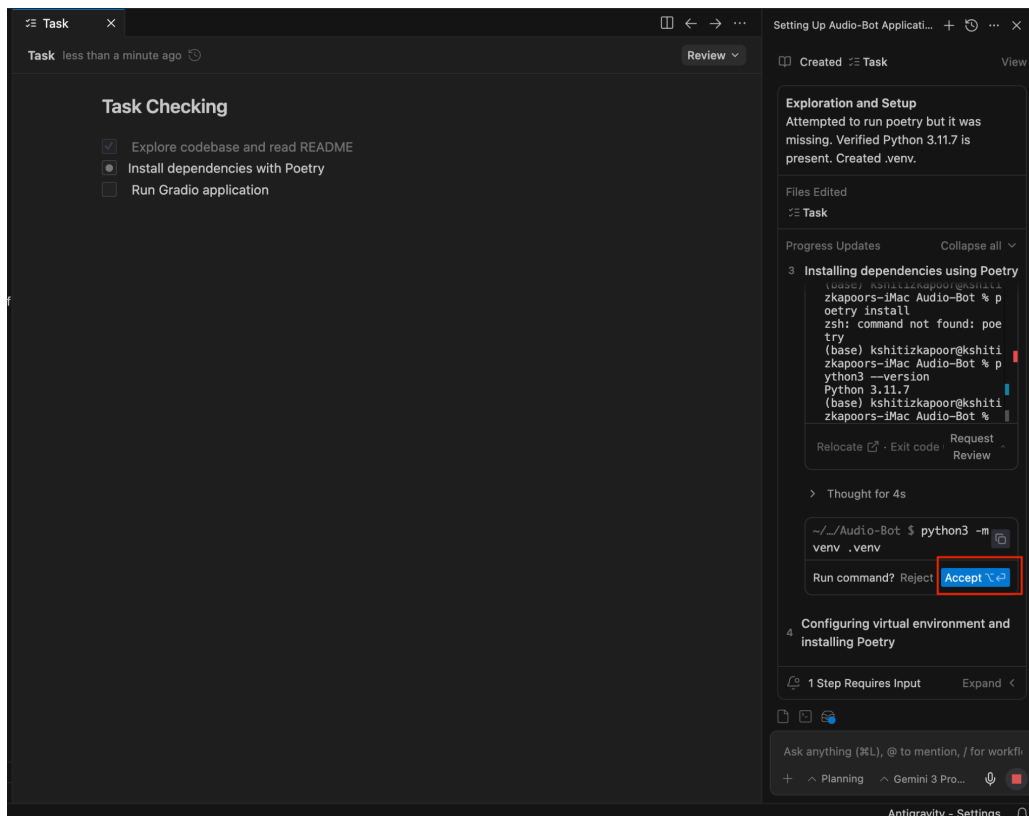


2. The agent will propose a plan, create a virtual environment, install dependencies and launch the Gradio interface, and ask you to review. Read through the plan; if something seems off, tweak your prompt and the agent will adjust.

Task: Explore and Launch Audio-Bot

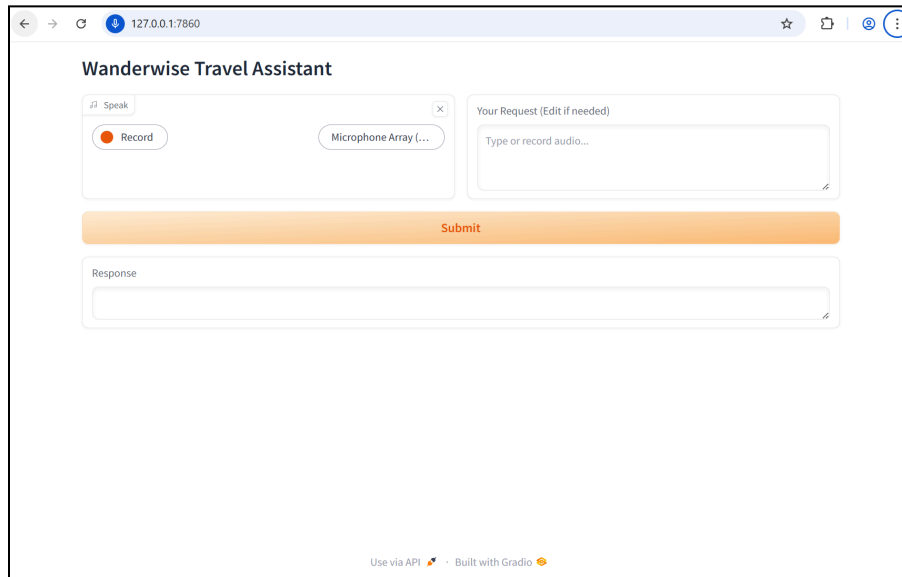
- ✓ Explore Project Structure
 - ✓ Read  `pyproject.toml`
 - ✓ List `app` directory
 - ✓ List `genai_voice` directory
 - ✓ Read key source files
- ✓ Setup Environment
 - ✓ Check/Create  `.env`
 - ✓ Verify dependencies
- ✓ Launch Project
 - ✓ Run `poetry run RunChatBotScript`

3. When the agent asks to run a command (like `poetry install`), click Approve. This ensures the environment is set up exactly as documented.



The screenshot displays the Antigravity AI interface. On the left, the 'Task Checking' section shows a list of tasks: 'Explore codebase and read README' (checked), 'Install dependencies with Poetry' (selected), and 'Run Gradio application' (unchecked). The main panel on the right shows the progress of the task. It includes a section for 'Exploration and Setup' with a message about Python 3.11.7 being verified and a `.venv` directory created. Below this, the 'Files Edited' section lists the task. The 'Progress Updates' section shows a list of steps, with the current step being 'Installing dependencies using Poetry'. This step includes a terminal log showing the command `poetry install` being executed, which resulted in a 'command not found' error. Below the log, there is a prompt 'Run command?' with a 'Reject' button and an 'Accept' button, which is highlighted with a red box. The bottom of the interface shows a chat input field with a placeholder 'Ask anything (%L), @ to mention, / for workf...' and a 'Planning' button.

4. Finally, once the agent has completed all the setup steps, Antigravity will automatically open the app in its built-in browser. From here, you can start interacting with the bot directly.



Interacting with the Itinerary Audio Bot

Once the Gradio app is running, Antigravity's Browser Preview panel shows the bot's interface. Try the following:

1. **Record your query** – Click the microphone icon and speak your travel request, e.g., “Plan a two-day trip to Goa with beach activities”. Make sure to grant microphone permission and record in a quiet room.
2. **Edit the transcript** – Whisper converts your speech to text. A text box lets you fix any transcription errors.
3. **Submit** – Hit **Submit**. The underlying language model uses context from `travel_bot_context.txt` in the data folder to generate a personalised itinerary; the response appears in the chat window.
4. **Iterate** – Ask follow-up questions or adjust your request (“Suggest vegetarian restaurants”). The model will respond accordingly.

Let's Implement a Feature: Giving the Bot a Voice!

Right now, our Travel Assistant is a bit... shy. It types everything out, but wouldn't it be cooler if it actually spoke to us?

In this section, we are going to perform a "Voice Transplant" on our bot. We will implement the code that turns text into lifelike speech.

The Mission

We want the bot to:

1. **Take the text** generated by the AI.
2. **Convert** it into audio.
3. **Play** it through your speakers.

We have prepared the skeleton code for you in **genai_voice/processing/audio.py** Your job is to fill in the missing 3 lines that make the magic happen!

Time to Code

Open **genai_voice/processing/audio.py** and scroll down to the **communicate** function. You'll see some comments marked **TODO**. That's where you come in.

Step 1: The Transformation

First, we need to convert the text string into audio data. We are using a library called **gTTS** (Google Text-to-Speech).

Look for **TODO 1** and add this line:

```
# Convert the text (phrase) into English audio
tts = gTTS(text=phrase, lang='en')
```

Step 2: The Storage

Computer audio players need a file to read. We can't just throw raw data at them! We need to save our new audio to a temporary file.

Look for **TODO 2** and add:

```
# Save the audio object to our temporary file path
tts.save(temp_file)
```

Step 3: The Performance

Finally, let's make some noise! We will use the winsound library to play the file we just created.

Look for **TODO 3** and add:

```
# Play the sound file immediately
winsound.PlaySound(temp_wav, winsound.SND_FILENAME)
```

What just happened?

- **gTTS**: This connects to Google's servers, sends your text, and gets back an MP3 file spoken by a standard AI voice.
- **save()**: Writes those bytes to your hard drive so other programs can access them.
- **winsound**: This is a built-in Windows tool that grabs an audio file and blasts it through your default speakers.

Test it out!

Save your file, run the bot, and say "Hello!". If you hear it reply, congratulations! You've just given your creation a voice. 🎉

Tips for Effective Prompt Engineering

Clear prompts help Antigravity's agents succeed. Remember:

- **Be specific but high-level** – Describe the desired outcome rather than listing every command. For example, "Install dependencies and run the Gradio app" is better than a long list of pip commands.
- **Reference documentation or errors** – Paste relevant snippets from the README or error messages into your prompt; the agent uses this context to make better decisions.
- **Iterate interactively** – Review-driven mode lets you check the agent's plan and correct it if needed. If the agent gets stuck, provide hints like "Use Poetry instead of pip" or ask it to run commands manually.
- **Use multiple agents** – You can create a second agent to work on a new feature (like TTS) while another runs the main app; the Agent Manager displays the status of each agent.

Troubleshooting and Best Practices

- **Audio quality matters:** use a headset microphone and record in a quiet room. Grant microphone permission in your browser.
- **Environment errors:** if you encounter missing library errors (e.g., pyaudio on macOS), install prerequisites (like portaudio on Mac) using your system package manager. The README may include additional instructions.
- **gTTS vs pyttsx3:** gTTS produces natural voices but needs an internet connection; pyttsx3 works offline and allows more customisation. Choose based on your deployment needs.
- **Security:** because Antigravity can run terminal commands and browse the web, keep the agent in review-driven mode so you must approve each action.

Conclusion

By combining the Itinerary Audio Bot with Google Antigravity, you'll experience the power of agent-first development. You watch an autonomous agent unpack a project, install dependencies and launch a Gradio app while maintaining control through review prompts. Adding text-to-speech makes the bot more accessible and engaging, whether you choose the cloud-based natural voices of gTTS or the flexibility of pyttsx3. With clear prompts, a willingness to iterate and an eye for experimentation, Antigravity becomes a playful yet powerful tool for learning generative AI.